Question 1: Consider a FICO pipeline with 4 stages:

IF DISP EX* SU

EX takes 2 (pipelined) cycles for LW, and 1 cycle for ADD. There is only one ADD unit and one LW unit. Two instructions per cycle are fetched by IF, and the DISP queue is unlimited in length.

Show the pipeline timing diagram for the following sequence of instructions as shown in class. You can assume that when as instruction completes and clears the busy bit on its destination register, any instruction, waiting in DISP for that register can fire in the same cycle.

A. LW R1, 0(R2)
B. ADD R3, R1, R2
C. ADD R5, R4, R2
D. LW R6, 0(R4)
E. ADD R7, R1, R5
F. ADD R8, R6, R3

Q #1									
	0	1	2	3	4	5	6	7	8
LW R1,0(R2)	IF	DISP	EX	EX	SU				
ADD R3, R1, R2	IF	DISP	DISP	DISP	EX	SU			
ADD R5, R4, R2		IF	DISP	DISP	DISP	EX	SU		
LW R6, 0(R4)		IF	DISP	DISP	DISP	EX	EX	SU	
ADD R7, R1, R5			IF	DISP	DISP	DISP	EX	SU	
ADD R8, R6, R3			IF	DISP	DISP	DISP	DISP	EX	SU

- **Highlight boxes where an instruction cannot make forward progress due to resource limitations.**

Question 2: The sequence in question 1 is now executed on an out-of-order processor that has six LW units and six ADD units. As before, LW is pipelined into 2 cycles, ADD is a single-cycle operation. The fetch rate is now 6 instructions/cycle. **Show the contents of the scheduling queue for traditional Tomasulo (no RAT/Pregs) and the register file** at two snapshots in time:

a) After all 6 instructions have been dispatched, and
b) After instructions A, C and D have completed.

Assume tags begin at the value 100. The sizes of CDBs and Scheduling Queue are unlimited. After an instruction completes and goes into State Update, we will update the RF of its destination register and any instructions waiting in SCHED for that register can fire in the same cycle. **Use the given tables for your final answer**. Note: You do not need to show the register values, just the tag, ready and register number fields of the scheduling queue.

a) After all 6 instructions have been dispatched

FU	Dest Reg	Dest Reg Tag	Src1 Ready	Src1 Tag	Src2 Ready	Src2 Tag
LW	R1	100	1		1	
ADD	R3	101	0	100	1	
ADD	R5	102	1		1	
LW	R6	103	1		1	
ADD	R7	104	0	100	0	102
ADD	R8	105	0	103	0	101

Register #	Ready	Tag
R1	0	100
R2	1	
R3	0	101
R4	1	
R5	0	102
R6	0	103
R7	0	104
R8	0	105
R9	1	
R10	1	

b) After instructions A, C and D have completed. (Cycle 5) (Highlighted Instructions have left the pipeline and need not be shown)

FU	Dest Reg	Dest Reg Tag	Src1 Ready	Src1 Tag	Src2 Ready	Src2 Tag
LW	R1	100	1		1	
ADD	R3	101	~~0~~ 1	100	1	
ADD	R5	102	1		1	
LW	R6	103	1		1	
ADD	R7	104	~~0~~ 1	100	~~0~~ 1	102
ADD	R8	105	~~0~~ 1	103	0	101

Register #	Ready	Tag
R1	~~0~~ 1	100
R2	1	
R3	0	101
R4	1	
R5	~~0~~ 1	102
R6	~~0~~ 1	103
R7	0	104
R8	0	105
R9	1	
R10	1	

Q #2									
	1	2	3	4	5	6	7	8	9
LW R1,0(R2)	IF	DISP	EX	EX	**SU**				
ADD R3, R1, R2	IF	DISP	DISP	DISP	EX	**SU**			
ADD R5, R4, R2	IF	DISP	EX	**SU**					
LW R6, 0(R4)	IF	DISP	EX	EX	**SU**				
ADD R7, R1, R5	IF	DISP	DISP	DISP	EX	**SU**			
ADD R8, R6, R3	IF	DISP	DISP	DISP	DISP	EX	**SU**		

Need not be explicitly shown

Question 3: The following instruction stream is executed on a superscalar pipeline that uses the PReg/RAT approach for register renaming. Show the state of the scheduling queue, register file and RAT after all the instructions have been dispatched:

A. LW R1, 0(R2)
B. ADD R3, R1, R2
C. ADD R5, R4, R2
D. LW R6, 0(R4)
E. ADD R7, R1, R5
F. ADD R8, R6, R3

You can assume that Pregs start from 0.

Scheduling Queue:

Sheduling Queue			
Functional Unit	Dest PReg	Src1 PReg	Src2 PReg
LW	P9		P2
ADD	P10	P9	P2
ADD	P11	P4	P2
LW	P12		P4
ADD	P13	P9	P11
ADD	P14	P12	P10

RAT	
Areg	Preg
R0	P0
R1	P9
R2	P2
R3	P10
R4	P4
R5	P11
R6	P12
R7	P13
R8	P14

Reg File	
Preg	Ready
P0-P8	1
P9	0
P10	0
P11	0
P12	0
P13	0
P14	0